

Solar System Simulator (C# Edition)

2D Solar System Physics Simulator

A Modular, Scene-Based Final-Year Project Roadmap (C# Edition)

1. Project Overview
 - 1.1 Vision
 - 1.2 Functional Goals
 - 1.3 Non-Goals (Explicitly Out of Scope)
 - 1.4 Target Audience
2. Library & Tooling Stack
 - 2.1 Core Rendering & UI
 - 2.2 Core Engine Infrastructure
 - 2.3 Testing & Profiling
 - 2.4 Optional / Stretch
 - 2.5 Build & Tooling
 - 2.6 Why Not These Other Libraries?
3. Architectural Foundation
 - 3.1 Module Boundaries
 - 3.2 ECS as the Spine
 - 3.3 Component Catalog
 - 3.4 Event Bus
 - 3.5 The Infinite Space Problem
 - 3.6 Threading Model
 - 3.7 Time Management
4. Project Structure
 - 4.1 Directory Layout
 - 4.2 Build Configuration
 - 4.3 Testing Strategy
5. Month-by-Month Roadmap
 - 5.1 Month 1: Foundation & Infrastructure
 - 5.1.1 Goal
 - 5.1.2 Tasks
 - 5.1.3 Implementation Notes
 - 5.1.4 Deliverable Demo
 - 5.1.5 Report Content
 - 5.2 Month 2: Physics Core v1
 - 5.2.1 Goal
 - 5.2.2 Tasks
 - 5.2.3 Integrator Math
 - 5.2.4 Deliverable Demo
 - 5.2.5 Report Content
 - 5.3 Month 3: Scene System, Serialization, Hierarchy
 - 5.3.1 Goal
 - 5.3.2 Tasks
 - 5.3.3 Implementation Notes
 - 5.3.4 Deliverable Demo
 - 5.3.5 Report Content
 - 5.4 Month 4: ImGui Editor — The Meat of Usability
 - 5.4.1 Goal
 - 5.4.2 Layout
 - 5.4.3 Interactions
 - 5.4.4 Undo/Redo (Critical)
 - 5.4.5 Inspector

- 5.4.6 Implementation Notes
- 5.4.7 Deliverable Demo
- 5.4.8 Report Content
- 5.5 Month 5: Particle System
 - 5.5.1 Goal
 - 5.5.2 Architecture
 - 5.5.3 Emitter Parameters
 - 5.5.4 Built-in Effect Presets
 - 5.5.5 Particle Editor UI
 - 5.5.6 Implementation Notes
 - 5.5.7 Deliverable Demo
 - 5.5.8 Report Content
- 5.6 Month 6: Barnes-Hut + Performance Scaling
 - 5.6.1 Goal
 - 5.6.2 Quadtree
 - 5.6.3 Barnes-Hut Force Calculation
 - 5.6.4 Multithreading
 - 5.6.5 Visualization
 - 5.6.6 Performance Comparison
 - 5.6.7 Implementation Notes
 - 5.6.8 Deliverable Demo
 - 5.6.9 Report Content
- 5.7 Month 7: Special Objects & Advanced Physics
 - 5.7.1 Goal
 - 5.7.2 Adaptive Timestep
 - 5.7.3 Black Holes
 - 5.7.4 Tidal Disruption
 - 5.7.5 Collision Modes
 - 5.7.6 Test Particles
 - 5.7.7 Visualization Overlays
 - 5.7.8 Implementation Notes
 - 5.7.9 Deliverable Demo
 - 5.7.10 Report Content
- 5.8 Month 8: Tools, Polish, Scenarios, Export
 - 5.8.1 Goal
 - 5.8.2 Orbit Designer
 - 5.8.3 Predicted Orbit Overlay
 - 5.8.4 Orbital Element Readout
 - 5.8.5 Lagrange Points
 - 5.8.6 Reference Frames
 - 5.8.7 Time Controls
 - 5.8.8 Export
 - 5.8.9 Scenario Library Expansion
 - 5.8.10 Onboarding & Settings
 - 5.8.11 Deliverable Demo
 - 5.8.12 Report Content
- 5.9 Month 9: Stretch Features (Pick 2-3)
 - 5.9.1 Recommended Picks (by effort/impact ratio)
 - 5.9.2 Lua Scripting (Strong Recommendation)
 - 5.9.3 Lensing Shader
 - 5.9.4 Deliverable Demo
 - 5.9.5 Report Content
- 5.10 Month 10: Report, Polish, Defense Prep
 - 5.10.1 Goal
 - 5.10.2 Code
 - 5.10.3 Documentation
 - 5.10.4 Report
 - 5.10.5 Defense Prep

6. Risk Register
7. What Makes This Defendable as a Final-Year Project
8. Mapping to CS Curriculum Criteria
9. C#-Specific Notes for the Learner
10. Final Note

2D Solar System Physics Simulator

A Modular, Scene-Based Final-Year Project Roadmap (C# Edition)

Tech Stack: C# 12 / .NET 8 LTS · Raylib-cs · ImGui.NET · ImPlot.NET · rllImGui-cs · Arch ECS · Silk.NET.Maths · custom particle system **Project Type:** Console Application (cross-platform: Windows + Linux + macOS via .NET) **Duration:** 8–10 months **Scope:** Medium–large, modular, scene-based gravitational simulator with infinite space

1. Project Overview

1.1 Vision

A modular, scene-based 2D gravitational physics simulator that lets users build solar systems, binary star systems, galactic collisions, and black-hole scenarios in an *infinite* 2D space. Users place bodies either directly into the scene viewport or via a scene hierarchy panel, edit them in an inspector, and watch real Newtonian (and post-Newtonian) physics play out — with a custom particle system providing visual richness (accretion disks, comet tails, tidal streams, nebulae, explosions).

The project is intentionally architected for **modularity**: the physics core, particle system, scene system, renderer, and UI are independent **assemblies** (separate `.csproj` projects in one `.sln`) connected via interfaces and an event bus. This makes the “modular architecture” claim measurable and defensible at viva — and it’s enforced at the linker/compile level, since `SolarSim.Core` simply doesn’t reference `Raylib-cs`.

1.2 Functional Goals

- **Scene-based authoring** — infinite 2D world; create/load/save scenes; multiple scenes per project.
- **Dual placement modes** — drag from a palette into the viewport, or build the hierarchy programmatically via the side panel.
- **Live physics simulation** — at least four numerical integrators with real-time switching; conservation diagnostics.
- **Scalable performance** — direct N-body for small scenes, Barnes-Hut for thousands of bodies; multithreaded via the Task Parallel Library.
- **Custom particle system** — pool-allocated, batched-rendered, capable of 100k+ particles at 60 fps without per-particle allocations.
- **Special objects** — stars (with corona), black holes (with event horizon, accretion disk, post-Newtonian gravity), test particles, asteroid belts.
- **Scenario library** — preset solar systems, three-body curiosities, galaxy collisions, tidal disruptions.
- **Export** — JSON scenes, CSV trajectory data, screenshots, video.
- **Plugin layer (stretch)** — Lua scripting via NLua for user-defined force laws, emitters, and behaviors.

1.3 Non-Goals (Explicitly Out of Scope)

Stating these in your report defends against scope-creep questions at viva:

- **3D rendering and physics.** 2D only. The same algorithms generalize but doubling the dimensionality doubles the work for no project-defining benefit.
- **General relativity.** First-order post-Newtonian correction only; full GR is a PhD topic.

- **Game-engine features.** No animation system, no scripting language design, no networking (stretch only).
- **Production polish.** This is a research/demonstration tool, not a commercial product. UI quality target: “clearly designed and pleasant to use,” not “ready to sell on Steam.”
- **Unity/Godot/MonoGame integration.** This is a standalone console application using raw raylib via Raylib-cs. We are not building inside a game engine.

1.4 Target Audience

- Astronomy/physics students wanting to visualize orbital mechanics intuitively.
- Educators teaching numerical methods (the integrator-comparison feature is genuinely pedagogical).
- Hobbyists who like simulation sandboxes (Universe Sandbox audience, but free and open).
- Examiners evaluating your final-year project. Be honest with yourself — they’re the primary audience.

2. Library & Tooling Stack

Pinning the library choices upfront because every architectural decision below assumes these. You should also pin **versions** in your `.csproj` files from day one — NuGet’s `<PackageReference Version="x.y.z" />` is your friend; version drift mid-project is a real and avoidable risk.

2.1 Core Rendering & UI

Library	NuGet ID	Purpose	Why this choice
Raylib-cs	<code>Raylib-cs</code>	Window, input, audio, 2D/3D rendering, shaders, image loading	Canonical C# binding for raylib; native bindings shipped in the NuGet, so no separate native install needed; well-maintained by ChrisDill
ImGui.NET	<code>ImGui.NET</code>	Immediate-mode GUI bindings (docking branch)	The standard ImGui binding for C#; maintained by mellinoe / community
rllmGui-cs	<code>rllmGui-cs</code>	Bridge between Raylib-cs and ImGui.NET	C# port of the official rllmGui binding; saves ~300 lines of boilerplate per app
ImPlot.NET	<code>ImPlot.NET</code>	High-performance plotting inside ImGui	The energy-conservation plots and performance graphs are non-negotiable; ImPlot is purpose-built for this

2.2 Core Engine Infrastructure

Library	NuGet ID	Purpose	Why this choice
Arch ECS	<code>Arch</code>	Entity Component System	Fastest mainstream archetype-based ECS in C#; struct components packed in contiguous chunks (cache-friendly); zero-allocation queries; well-documented
Silk.NET.Maths	<code>Silk.NET.Maths</code>	Generic vector/matrix types (<code>Vector2D<double></code> !)	<code>System.Numerics.Vector2</code> is float-only — useless for double-precision world coordinates. Silk.NET.Maths provides <code>Vector2D<T></code> where T can be <code>double</code>
System.Text.Json	(built-in)	JSON serialization for scenes	The de-facto modern JSON library; built into .NET 8; faster and lower-allocation than Newtonsoft.Json
Serilog	<code>Serilog</code> + <code>Serilog.Sinks.Console</code> + <code>Serilog.Sinks.File</code>	Structured logging	The most-used logger in the modern .NET ecosystem; clean fluent API; structured log events

2.3 Testing & Profiling

Library	NuGet ID	Purpose	Why this choice
xUnit	<code>xunit</code> + <code>xunit.runner.visualstudio</code>	Unit test framework	The de-facto modern .NET test framework; first-class IDE and <code>dotnet test</code> support
FluentAssertions	<code>FluentAssertions</code>	Assertion library	Makes test failures readable: <code>result.Should().BeApproximately(expected, 1e-6)</code>
BenchmarkDotNet	<code>BenchmarkDotNet</code>	Micro-benchmarking	Industry-standard for .NET microbenchmarks; statistically rigorous; will be invaluable for month 6
JetBrains dotTrace	(commercial; free for students)	Frame profiler	Best graphical profiler for .NET on Windows + Linux. Alternative: <code>dotnet-trace</code> + PerfView (free)

2.4 Optional / Stretch

Library	NuGet ID	Purpose	When to add
NLua	<code>NLua</code>	Lua scripting	Month 9 if doing the scripting layer; binds against KeraLua; works on all platforms
MoonSharp	<code>MoonSharp</code>	Pure-C# Lua interpreter (alternative to NLua)	If NLua's native dependency causes deployment friction; trades speed for portability
ImageSharp	<code>SixLabors.ImageSharp</code>	PNG/JPG export and processing	Month 8 if you need frame export and don't want to rely on raylib's image export alone
NAudio	<code>NAudio</code> (Windows only) — or raylib's audio	Audio sonification	Month 9 if doing audio (raylib's <code>PlaySound</code> covers most needs; only reach for NAudio if you need real-time synthesis)

2.5 Build & Tooling

Tool	Purpose	Notes
.NET 8 SDK (LTS)	Build, test, run	LTS until November 2026 — safely covers your entire project timeline. Avoid .NET 9 (STS) and .NET 10 (preview during your project) to dodge breaking changes.
<code>dotnet</code> CLI	Build, restore, test, publish	All you need; works identically on Windows, Linux, macOS
MSBuild + <code>.csproj</code>	Build configuration	Modern SDK-style projects; one file per project, ~10 lines each
<code>Directory.Packages.props</code>	Centralized NuGet version pinning	Define every NuGet version in one place; reference in projects without versions. Prevents version drift across projects in your solution
<code>.editorconfig</code>	Code formatting + analyzer rules	Pin a <code>.editorconfig</code> in repo root from day one. <code>dotnet format</code> enforces it
Roslyn analyzers	Static analysis	<code><EnableNETAnalyzers>true</EnableNETAnalyzers></code> in your projects; auto-enabled in .NET 8
DocFX	API documentation	Generates HTML docs from XML doc comments; useful for the report appendix
GitHub Actions	Continuous integration	Build + run tests on every push; the Actions ecosystem has first-class .NET support

2.6 Why Not These Other Libraries?

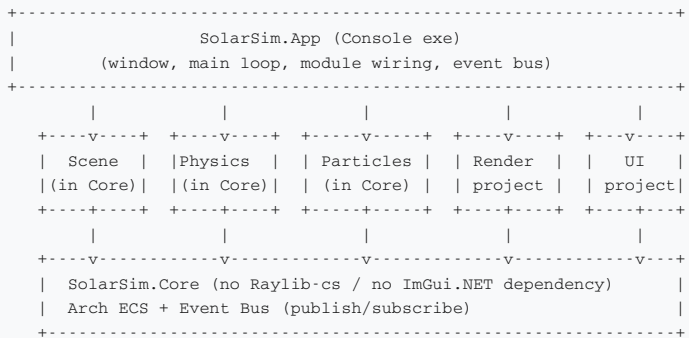
Anticipating the “why didn’t you use X?” viva question:

- **MonoGame / FNA** — full game frameworks with their own content pipeline and Game class. We need a clean console app with raylib; MonoGame would impose architectural assumptions that don’t fit.
- **Unity / Godot** — heavyweight engines; the project would become “a Unity simulation” rather than a from-scratch implementation. The “I implemented Barnes-Hut myself” claim weakens.
- **OpenTK** — bare OpenGL bindings; we’d be reimplementing raylib’s window/input/audio. Wrong abstraction level.
- **Siik.NET** (the full bindings library, not just Maths) — much lower-level than raylib; great for serious graphics work, overkill here.
- **Avalonia / WPF / WinForms** — retained-mode UI toolkits; the immediate-mode ImGui model is a far better fit for tooling UI with continuously-changing state.

- **DefaultEcs / Friflo.Engine.ECS** — both are excellent alternatives to Arch. Friflo benchmarks ~2× faster than Arch in some scenarios but is currently in preview (3.0.0-preview). DefaultEcs is sparse-set based; slower for our access patterns. Arch is the stable, fast, well-known choice.
- **REBOUND** (the academic N-body library) — would do all the physics for you, defeating the point of the project. You're *implementing* the algorithms, not consuming them.

3. Architectural Foundation

3.1 Module Boundaries



The **non-negotiable rule**: `SolarSim.Core` does not reference `Raylib-cs` or `ImGui.NET`. The `.csproj` files enforce this. If you accidentally add `using Raylib-cs;` inside `SolarSim.Core`, the build fails. Mechanical guarantee beats documentation every time.

Concretely:

- `SolarSim.Core` references only `Arch`, `Silk.NET.Maths`, `Serilog`, `System.Text.Json`. No raylib, no ImGui.
- `SolarSim.Render` references `SolarSim.Core` + `Raylib-cs`.
- `SolarSim.UI` references `SolarSim.Core` + `ImGui.NET` + `ImPlot.NET` + `Raylib-cs` (for `rlImGui-cs`).
- `SolarSim.App` references all of the above; this is the only project where everything comes together.
- All cross-module communication goes through (a) the ECS world (`Arch.World`), (b) the event bus, or (c) explicit interfaces (`IIntegrator`, `IForceField`).

Examiners *will* check by asking “if I want to swap your integrator, how many files do I touch?”

3.2 ECS as the Spine

For a scene-based tool with thousands of entities (bodies + particles + UI gizmos), an Entity Component System is the right pattern. Arch’s specific advantages for our use case:

- **Decoupling** — components are `struct`s (pure data); systems are query iterations (pure behavior); no inheritance hierarchy to manage.
- **Cache coherence** — Arch stores components of the same type in contiguous chunks within archetypes. Iterating “all entities with `Transform + RigidBody`” is essentially walking two parallel arrays.
- **Composability** — want a “body that’s also a particle emitter”? Just attach both components to the entity.
- **Zero-allocation iteration** — Arch’s `Query` API uses `ref struct` enumerators and `ref T` returns. Properly written queries allocate nothing per frame.
- **Serializability** — components are POD-style structs, so `System.Text.Json` serialization is mostly mechanical.

C# note on `struct` vs `class`: components must be `struct` (value types). Class components would force heap allocation per entity and destroy cache coherence. We use `struct` everywhere for components, `class` only for systems/services.

3.3 Component Catalog

The full list of components you’ll define. Pin this in `Components.cs` early; resist the urge to keep adding new ones.

```

using Silk.NET.Maths;
using Raylib_cs;
using System.Collections.Generic;

namespace SolarSim.Core.Scene;

public struct Transform
{
    public Vector2D<double> Position; // double precision for infinite world
    public double Rotation;          // radians
}

public struct RigidBody
{
    public Vector2D<double> Velocity;
    public Vector2D<double> Acceleration;
    public Vector2D<double> ForceAccumulator;
    public double Mass;
    public double InverseMass; // cached; 0 for fixed bodies
    public bool IsTestParticle; // feels gravity but doesn't exert it
    public bool IsFixed; // can't move (e.g., scene anchor)

    public void SetMass(double mass)
    {
        Mass = mass;
        InverseMass = mass > 0.0 ? 1.0 / mass : 0.0;
    }
}

public struct Renderable
{
    public Color Color; // raylib Color is just 4 bytes; safe to reference here
    public float RadiusPixels; // visual radius (independent of physical)
    public int RenderLayer;
    public bool Visible;
}

public struct Trail
{
    public Queue<Vector2D<double>> Points; // ring buffer in practice
    public int MaxPoints;
    public Color StartColor;
    public Color EndColor;
    public float Width;
}

public struct BlackHole
{
    public double SchwarzschildRadius;
    public bool ShowPhotonSphere;
    public bool EnableLensingShader;
}

public struct ParticleEmitter
{
    public EmitterShape Shape;
    public float RatePerSecond;
    public Range<double> InitialSpeed;
    public Range<double> InitialAngle;
    public Range<double> Lifetime;
    public ColorGradient ColorOverLife;
    public Curve SizeOverLife;
    public BlendMode Blend;
    public bool AffectsGravity;
    public bool AffectedByGravity;
}

public struct Hierarchy
{
    public int ParentId; // -1 = no parent (we map entities to ints in JSON)
    public List<int> ChildIds;
}

public struct Name { public string Value; }
public struct Tag { public string Value; }

public struct OrbitPredictor
{
    public List<Vector2D<double>> ProjectedPath;
}

```



```

public int SampleCount;           // 200 default
public double HorizonSeconds;     // 1 year default
public bool NeedsRecompute;
}

```

Note on `Color`: raylib's `Color` is a 4-byte struct (rgba). Strictly speaking, having `Renderable` reference a raylib type leaks the dependency into Core. The pragmatic choice is to accept this small leak (it's a POD-style 4-byte struct, not an opinionated API surface). The purist alternative is a `SolarSim.Core.Color` type that the renderer converts. Document the choice in your report; either is defensible.

3.4 Event Bus

A simple type-keyed publish/subscribe system. Sketch:

```

public class EventBus
{
    private readonly Dictionary<Type, List<Delegate>> _handlers = new();

    public void Subscribe<TEvent>(Action<TEvent> handler)
    {
        var type = typeof(TEvent);
        if (!_handlers.TryGetValue(type, out var list))
            _handlers[type] = list = new List<Delegate>();
        list.Add(handler);
    }

    public void Publish<TEvent>(in TEvent evt)
    {
        if (!_handlers.TryGetValue(typeof(TEvent), out var list)) return;
        foreach (var d in list) ((Action<TEvent>)d)(evt);
    }
}

```

Events to define from the start (as `record struct` for zero-allocation publishing):

- `BodyCreatedEvent`, `BodyDestroyedEvent`
- `BodyCollisionEvent(Entity A, Entity B, Vector2D<double> ContactPoint, double ImpactEnergy)`
- `BodySelectedEvent`, `BodyDeselectedEvent`
- `SceneLoadedEvent`, `SceneSavedEvent`, `SceneClearedEvent`
- `IntegratorChangedEvent`, `TimescaleChangedEvent`
- `BlackHoleAbsorbedBodyEvent`
- `RocheLimitCrossedEvent` (for tidal disruption trigger)

The whole thing is ~100 lines of C#. It pays for itself immediately because particles, audio, UI, and renderer all want to react to physics events without being coupled to physics.

3.5 The Infinite Space Problem

"Infinite" in practice means three things:

Double-precision world coordinates. Positions and velocities are `Vector2D<double>` (from `Silk.NET.Maths`). `System.Numerics.Vector2` is float-only and gives ~7 decimal digits — at solar-system scales (10^8 km) with small visible radii ($\sim 10^3$ km) you'll see jittering after a few minutes. Doubles give 15-16 digits, which is enough for centuries of simulation without catastrophic precision loss. *This is the single most important reason to depend on `Silk.NET.Maths` rather than `System.Numerics`.*

Floating origin. When the camera is far from world origin (say, after panning to follow a body 10^{12} units away), the *render-time* coordinates passed to OpenGL are still floats. Accumulated float error at that distance becomes visible. Solution: when the camera drifts past a threshold, subtract the camera position from every entity's render coordinate at draw time, so the renderer always works near zero. The simulation state stays in absolute world coordinates.

Logarithmic zoom. Zoom level should be exponential — each scroll-wheel notch multiplies zoom by ~1.2x. Linear zoom is unusable when ranging from "see Pluto's orbit" (10^{13} m) to "see craters on Mercury" (10^6 m). Implement zoom as `worldPerPixel = base * Math.Pow(zoomFactor, zoomLevel)`.

3.6 Threading Model

For a final-year project, simple is better than clever. Recommended layout, leveraging .NET's TPL (Task Parallel Library):

- **Main thread:** input, UI, rendering, scene mutation. Owns the canonical Arch `World`.
- **Physics thread (optional, later):** ticks the simulation in a fixed loop, double-buffered state read by main thread for rendering. Skip in v1; add in month 6 if needed.
- **Worker pool** (month 6): `Parallel.For` or hand-rolled `Task` partitioning across the body array during Barnes-Hut force calculation. .NET's `ThreadPool` handles the scheduling; you don't need to manage worker threads explicitly.

Critical rule: **do not share mutable ECS state across threads without synchronization**. Either snapshot before parallel work, or partition the work so threads operate on disjoint slices (Barnes-Hut is naturally embarrassingly parallel — each body's force calculation is independent given a read-only tree).

C# note on `async`: the simulation loop is not `async`. `Async` is for I/O concurrency; we want CPU concurrency, which `Task.Run` + `Parallel.For` provide. Don't be tempted to make your physics loop `async` — it'll just slow it down.

3.7 Time Management

Use **fixed-timestep simulation with interpolated rendering** (Glenn Fiedler, "Fix Your Timestep"). Sketch:

```
const double dt = 1.0 / 60.0;           // simulation step (seconds, scaled)
double accumulator = 0.0;
double timeScale = 1.0;                 // user-controllable (0.1x to 1000x)

var stopwatch = Stopwatch.StartNew();
double lastTime = 0.0;

while (!Raylib.WindowShouldClose())
{
    double currentTime = stopwatch.Elapsed.TotalSeconds;
    double frameTime = Math.Min(currentTime - lastTime, 0.25); // clamp to avoid spiral-of-death
    lastTime = currentTime;

    accumulator += frameTime * timeScale;

    while (accumulator >= dt)
    {
        physicsWorld.Step(dt);
        accumulator -= dt;
    }

    double alpha = accumulator / dt;      // for render interpolation
    renderer.Draw(scene, alpha);
}
```

Why fixed timestep: numerical stability of integrators depends on consistent `dt`. Variable timestep introduces error correlated with frame rate, which is bad for both reproducibility and physics correctness.

C# note on `Stopwatch`: use `Stopwatch.Elapsed.TotalSeconds`, not `DateTime.UtcNow` — `DateTime` has ~16ms resolution on Windows, useless for frame timing. `Stopwatch` uses the high-resolution performance counter.

4. Project Structure

4.1 Directory Layout

```
solar-sim/
├── SolarSim.sln           # solution file
├── Directory.Packages.props # centralized NuGet versions
├── Directory.Build.props  # shared MSBuild settings (LangVersion, Nullable, etc.)
├── README.md
├── LICENSE
├── .editorconfig
└── .gitignore
```

```

└─ .github/workflows/ci.yml
└─ src/
    └─ SolarSim.Core/
        └─ SolarSim.Core.csproj
        └─ Logging/
            └─ LoggerFactory.cs
        └─ Events/
            └─ EventBus.cs
            └─ Events.cs          # all event record-structs
        └─ Math/
            └─ MathUtils.cs
        └─ Scene/
            └─ Scene.cs          # wraps Arch.World
            └─ SceneManager.cs
            └─ Components.cs
            └─ EntityFactory.cs
            └─ Serialization/
                └─ SceneSerializer.cs
                └─ ComponentConverters.cs
        └─ Physics/
            └─ PhysicsWorld.cs
            └─ Integrators/
                └─ IIntegrator.cs
                └─ EulerIntegrator.cs
                └─ SymplecticEulerIntegrator.cs
                └─ VerletIntegrator.cs
                └─ Rk4Integrator.cs
            └─ ForceFields/
                └─ IForceField.cs
                └─ GravityField.cs
                └─ BarnesHutField.cs
            └─ Quadtree.cs
            └─ Collisions.cs
        └─ Particles/
            └─ Particle.cs
            └─ ParticlePool.cs
            └─ ParticleSystem.cs
            └─ EmitterPresets.cs
            └─ Noise.cs          # Perlin/curl

    └─ SolarSim.Render/
        └─ SolarSim.Render.csproj
        └─ Camera2D.cs          # double-precision world camera (NOT raylib's Camera2D)
        └─ WorldRenderer.cs
        └─ TrailRenderer.cs
        └─ OverlayRenderer.cs   # quadtree viz, gizmos
        └─ ParticleRenderer.cs  # batched/instanced
        └─ Shaders/
            └─ lensing.fs
            └─ particle.vs / .fs

    └─ SolarSim.UI/
        └─ SolarSim.UI.csproj
        └─ Editor.cs           # top-level UI orchestrator
        └─ Panels/
            └─ IPanel.cs
            └─ HierarchyPanel.cs
            └─ InspectorPanel.cs
            └─ ViewportPanel.cs
            └─ ToolbarPanel.cs
            └─ DiagnosticsPanel.cs
            └─ ParticleEditorPanel.cs
        └─ Commands/           # undo/redo
            └─ ICommand.cs
            └─ CommandStack.cs
            └─ CreateEntityCommand.cs
            └─ DestroyEntityCommand.cs
            └─ ModifyComponentCommand.cs
            └─ SelectionState.cs

    └─ SolarSim.App/
        └─ SolarSim.App.csproj  # <OutputType>Exe</OutputType>
        └─ Program.cs          # entry point
        └─ Application.cs      # main loop, module wiring

└─ tests/
    └─ SolarSim.Core.Tests/
        └─ SolarSim.Core.Tests.csproj
        └─ Physics/
            └─ IntegratorTests.cs

```

```

├── QuadtreeTests.cs
├── ConservationTests.cs
├── Scene/
├── SerializationTests.cs
├── Particles/
├── PoolTests.cs
├── SolarSim.Benchmarks/ # BenchmarkDotNet project
├── SolarSim.Benchmarks.csproj
├── BarnesHutBenchmarks.cs
├── assets/
├── fonts/
├── textures/
├── shaders/
├── scenarios/
├── solar_system.json
├── binary_stars.json
├── figure_eight.json
├── pythagorean.json
├── galaxy_collision.json
├── tidal_disruption.json
├── docs/
├── architecture.md
├── user_manual.md
├── images/

```

4.2 Build Configuration

`SolarSim.sln` references 6 projects:

- `SolarSim.Core` (class library) — physics, scene, particles. **No Raylib-cs / ImGui.NET dependency.**
- `SolarSim.Render` (class library) — depends on Raylib-cs + Core.
- `SolarSim.UI` (class library) — depends on ImGui.NET + ImPlot.NET + rllimgui-cs + Core.
- `SolarSim.App` (executable, `<OutputType>Exe</OutputType>`) — wires everything together.
- `SolarSim.Core.Tests` (xUnit) — depends on Core only.
- `SolarSim.Benchmarks` (BenchmarkDotNet) — depends on Core only.

This split **enforces the modularity claim at the linker level**. If `SolarSim.Core` accidentally pulls in `Raylib-cs`, the build breaks. Mechanical guarantee beats documentation every time.

`Directory.Packages.props` at solution root pins every NuGet version centrally:

```

<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="Raylib-cs" Version="6.1.1" />
    <PackageVersion Include="ImGui.NET" Version="1.91.6.1" />
    <PackageVersion Include="ImPlot.NET" Version="0.16.0" />
    <PackageVersion Include="rllimgui-cs" Version="2.1.0" />
    <PackageVersion Include="Arch" Version="1.2.8" />
    <PackageVersion Include="Silk.NET.Maths" Version="2.23.0" />
    <PackageVersion Include="Serilog" Version="4.0.2" />
    <PackageVersion Include="Serilog.Sinks.Console" Version="6.0.0" />
    <PackageVersion Include="Serilog.Sinks.File" Version="6.0.0" />
    <PackageVersion Include="xunit" Version="2.9.2" />
    <PackageVersion Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageVersion Include="FluentAssertions" Version="7.0.0" />
    <PackageVersion Include="BenchmarkDotNet" Version="0.14.0" />
  </ItemGroup>
</Project>

```

Why central package management: four projects each pinning `Arch` versions independently is a recipe for “feature works in tests but not in app” mysteries. Pin versions in one place.

Verify these versions against current NuGet at the moment you start the project; bump as needed but commit the bump.

4.3 Testing Strategy

- **Unit tests** — math utilities, integrators (analytic test cases), quadtree construction, serialization round-trips.
- **Property tests** — energy conservation under symplectic integrators (within tolerance), momentum conservation (machine precision), Barnes-Hut converges to direct sum as $\theta \rightarrow 0$.
- **Regression tests** — load each shipped scenario, run for N steps, compare end state to recorded baseline (hash of all positions). Catches accidental physics changes.
- **Visual tests** — manually run “demo scenarios” before each milestone; record screenshots for the report.
- **Benchmarks** — separate `SolarSim.Benchmarks` project using BenchmarkDotNet. Run quarterly (months 6, 8, 10) to catch perf regressions.

Target: 60-70% line coverage on `SolarSim.Core`. Don’t test rendering or UI exhaustively — too brittle.

5. Month-by-Month Roadmap

Each month has a **Goal**, a detailed **Task list**, **Implementation notes** for the tricky parts, a **Deliverable demo**, and **Report content** to write while it’s fresh in your head.

5.1 Month 1: Foundation & Infrastructure

5.1.1 Goal

A window opens; ECS works; pan/zoom camera works; one circle drifts across an infinite grid; tests run in CI.

5.1.2 Tasks

- Install **.NET 8 SDK**; verify with `dotnet --version`.
- Create GitHub repo with branch protection on `main`.
- `dotnet new sln -n SolarSim`; create projects via `dotnet new classlib`, `dotnet new console`, `dotnet new xunit`.
- Wire project references via `dotnet add reference`.
- `Directory.Packages.props` with pinned NuGet versions (\$4.2).
- `Directory.Build.props` with `<LangVersion>12</LangVersion>`, `<Nullable>enable</Nullable>`, `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>`.
- `.editorconfig` for formatting; verify `dotnet format` passes.
- GitHub Actions CI: build on Ubuntu + Windows, run tests, fail on warnings.
- **Application shell (`SolarSim.App/Program.cs` + `Application.cs`):**
 - Window creation via `Raylib.InitWindow(...)`.
 - Main loop with fixed timestep + interpolation (\$3.7).
 - rlImGui setup: `rlImGui.Setup(darkTheme: true)` + `rlImGui.Begin()` / `rlImGui.End()` in the loop.
 - ImGui docking flags: `ImGui.GetIO().ConfigFlags |= ImGuiConfigFlags.DockingEnable`.
 - Serilog initialization writing to console + rolling file in `logs/`.
- `EventBus` implementation + unit tests.
- `Scene` class wrapping `Arch.World`.
- Components: `Transform`, `RigidBody`, `Renderable`, `Name` (full catalog can wait).
- `Camera2D` class with double-precision world coordinates, pan via right-mouse-drag, zoom via scroll, world↔screen transforms.
- Simple debug grid renderer (lines every N world units).
- Hardcoded test entity: a circle with `Transform + RigidBody + Renderable + Name`, given an initial velocity, drifting visibly.
- One unit test passing (event bus delivers events to subscribers).
- README with build instructions: `dotnet build`, `dotnet run --project src/SolarSim.App`.

5.1.3 Implementation Notes

- **Coordinate system** — choose +Y up to match physics convention. Convert at render time, since raylib’s screen space is +Y down.

- **Fixed timestep** — pick `dt = 1.0/60.0` simulation seconds and a `timeScale` UI control. Don't tie `dt` to frame rate.
- **Don't fight `System.Numerics`** — use `Silk.NET.Maths.Vector2D<double>` for world coordinates, accept `System.Numerics.Vector2` (float) at the raylib boundary; convert explicitly. Cast helpers in `MathUtils.cs`.
- **Don't over-engineer the camera yet** — simple pan/zoom is enough; follow-body and rotating-frame come in month 8.
- **Naming convention** — PascalCase for public members (C# standard), `_camelCase` for private fields. Stick to it from day one.

5.1.4 Deliverable Demo

`dotnet run --project src/SolarSim.App` opens a window, you see a grid stretching to infinity, you pan with right-click drag, zoom with the scroll wheel, watch a small circle drift across the screen. Resize the window — UI stays sane. Open ImGui's demo window (`ImGui.ShowDemoWindow()`) just to confirm the UI integration works.

5.1.5 Report Content

- Architecture chapter draft: module boundaries diagram, dependency rationale, project structure.
- Why ECS? (data-oriented design, cache coherence, decoupling — cite Arch documentation and the [Game Programming Patterns](#) chapter on Component).
- Why fixed timestep? (cite Fiedler's article).
- Why .NET 8 LTS over .NET 9/10? (lifecycle alignment with project duration).

5.2 Month 2: Physics Core v1

5.2.1 Goal

Bodies actually orbit using real numerical integration. Multiple integrators are runtime-switchable. Energy and momentum are tracked and plotted.

5.2.2 Tasks

- Define `IIntegrator` interface:

```
public interface IIntegrator
{
    void Step(World world, double dt);
    string Name { get; }
    bool IsSymplectic { get; }
    bool IsTimeReversible { get; }
}
```

- Implement four integrators (see §5.2.3 below).
- Force accumulator pattern: each frame, `IForceField` implementations push forces into `RigidBody.ForceAccumulator`; integrator consumes and resets it.
- Implement `GravityField` (direct $O(N^2)$ summation) as the first force field.
- `PhysicsWorld` class owns the active integrator and the list of force fields; calls them in order each step.
- Conservation tracking:
 - Total kinetic energy: $\sum \frac{1}{2} m_i v_i^2$.
 - Total potential energy: $-\sum_{i < j} G m_i m_j / r_{ij}$ (note the $i < j$, don't double-count).
 - Linear momentum: $\sum m_i v_i$ (a 2D vector).
 - Angular momentum about origin: $\sum m_i (x_i v_y - y_i v_x)$.
 - Sample these every N frames (configurable, default every 10).
- `DiagnosticsPanel` (ImPlot.NET): four time-series plots, scrolling window.
- Hardcoded scenario in code: Sun + Earth + Mars + Jupiter, real masses, real orbital radii, computed circular-orbit velocities.
- `Trail` component + trail renderer (ring buffer of last N positions, fading line).
- Unit tests (xUnit + FluentAssertions):
 - Two-body circular orbit stays circular within 1% over 10 orbits (Verlet).

- Two-body energy conservation tolerance check across all integrators.
- Momentum exactly conserved to machine precision (since it's just a sum).
- Pythagorean three-body initial state matches published reference.

5.2.3 Integrator Math

Given body state (x, v) and acceleration $a(x) = F(x)/m$:

Explicit Euler — first-order, leaks energy fast:

```
v_new = v + a(x) * dt
x_new = x + v * dt
```

Symplectic (semi-implicit) Euler — first-order but symplectic, much better energy behavior:

```
v_new = v + a(x) * dt
x_new = x + v_new * dt      (note: uses v_new, not v)
```

Velocity Verlet — second-order, symplectic, time-reversible. The workhorse:

```
x_new = x + v * dt + 1/2 * a(x) * dt^2
a_new = a(x_new)      (recompute forces at new position)
v_new = v + 1/2 * (a(x) + a_new) * dt
```

RK4 — fourth-order, NOT symplectic (secretly leaks energy on long runs, despite higher accuracy per step):

```
k1 = f(t, y)
k2 = f(t + dt/2, y + dt/2 * k1)
k3 = f(t + dt/2, y + dt/2 * k2)
k4 = f(t + dt, y + dt * k3)
y_new = y + dt/6 * (k1 + 2k2 + 2k3 + k4)
```

where $y = (x, v)$ and $f(t, y) = (v, a(x))$.

The pedagogical demo: run a chaotic three-body system. RK4 visibly leaks total energy; Verlet stays stable. This single demo earns you a chapter in the report.

C# performance note: Arch's `Query.ForEach((ref Transform t, ref RigidBody r) => { ... })` pattern uses `ref` returns for zero-copy access. Use this consistently in integrator hot loops. Do not use `World.Get<Transform>()` repeatedly in a loop — that's the slow path.

5.2.4 Deliverable Demo

Mini solar system orbits stably for hours of wall-clock time. Drop down the integrator selector → switch from Verlet to RK4 → watch the energy plot start drifting. Switch to Euler → watch it crash within minutes.

5.2.5 Report Content

- Numerical methods chapter — derivations of all four integrators, geometric interpretation of symplecticity (preservation of phase-space volume), error analysis (local truncation error vs. global energy drift).
- Empirical results: error vs. dt log-log plots for each integrator on the two-body problem; energy drift over 1000 orbits.
- Conservation law derivations (where they come from in the Lagrangian; why momentum is exactly conserved by any pairwise-force integrator).

5.3 Month 3: Scene System, Serialization, Hierarchy

5.3.1 Goal

Create, save, load scenes. Entities have parent-child relationships visible in a UI tree. Multiple scenarios ship with the build.

5.3.2 Tasks

- `Scene` class with metadata (name, author, simulation settings, camera state, version).
- `SceneManager` — owns active scene, can swap, fires `SceneLoadedEvent` / `SceneClearedEvent`.
- `Hierarchy` component (parent + children list).
- Note: hierarchy is *organizational* (UI grouping). Physics doesn't care about hierarchy — gravity acts on world positions regardless of parent.
- Optional: support transform inheritance for moons (child position is relative to parent). Decision: defer to month 8 unless trivial.
- Per-component serialization using `System.Text.Json` :
 - For complex types (`Vector2D<double>` , `Color`), write `JsonConverter<T>` classes once.
 - For component structs themselves, source-generated serialization (`[JsonSerializable(typeof(Transform))]`) gives zero-reflection, AOT-safe JSON. Recommended for performance.
 - Handle entity references (for `Hierarchy.ParentId`) by mapping to integer IDs in the JSON.
- `SceneSerializer.Save(scene, path)` , `SceneSerializer.Load(path)` .
- File format with explicit `version` field. Bump it any time you break backward compatibility; write a migration step for old files in `SceneSerializer.Migrate(jsonNode, fromVersion, toVersion)` .
- `EntityFactory` — wraps the boilerplate of attaching the right components for common archetypes:
 - `factory.CreatePlanet(name, mass, position, velocity, color)`
 - `factory.CreateStar(name, mass, position, velocity, luminosity)`
 - `factory.CreateBlackHole(name, mass, position)`
 - `factory.CreateTestParticle(position, velocity)`
- Six scenarios as JSON files in `scenarios/` :
 - `solar_system.json`
 - `binary_stars.json`
 - `figure_eight.json`
 - `pythagorean.json`
 - `earth_moon.json`
 - `inner_planets.json`
- Scene browser UI (a simple list with thumbnails — placeholder until month 8).
- Tests: round-trip serialization (save → load → compare entity-by-entity using `FluentAssertions` 's `BeEquivalentTo`).

5.3.3 Implementation Notes

Scene file format example:

```
{
  "version": 2,
  "name": "Inner Solar System",
  "camera": { "position": [0, 0], "zoom": 1.0 },
  "physics": { "integrator": "verlet", "timescale": 1.0 },
  "entities": [
    {
      "id": 1,
      "components": {
        "Transform": { "position": [0, 0], "rotation": 0 },
        "RigidBody": { "velocity": [0, 0], "mass": 1.989e30 },
        "Renderable": { "color": [255, 220, 80, 255], "radius": 20 },
        "Name": { "value": "Sun" }
      }
    }
  ]
}
```

- **Don't serialize derived state:** things like `ForceAccumulator` and `InverseMass` should not be in JSON; recompute on load.
- **Stable IDs:** assign sequential integer IDs in JSON. Don't try to round-trip Arch's `Entity.Id` values; they're not meaningful across runs.
- **Two-pass load:** pass 1 creates all entities and builds an `id → Entity` map; pass 2 deserializes components (so

`Hierarchy.ParentId` can be resolved through the map).

- `System.Text.Json` **gotcha**: by default it's case-insensitive for property names but case-sensitive for dictionary keys. Set `PropertyNamingPolicy = JsonNamingPolicy.CamelCase` consistently.

5.3.4 Deliverable Demo

Click File → Load → “Inner Solar System” → planets appear, orbit. File → Save As → `my_scene.json` → close app → reopen → load `my_scene.json` → state restored exactly.

5.3.5 Report Content

- Scene system chapter: design rationale, hierarchy semantics, versioning strategy.
 - Serialization design: trade-offs of JSON vs. binary, why `System.Text.Json` source generators over `Newtonsoft.Json` reflection.
-

5.4 Month 4: ImGui Editor — The Meat of Usability

5.4.1 Goal

Stop being a tech demo, start being a tool. Click-to-create, drag-to-edit, undo/redo, save/load all wired together.

5.4.2 Layout

ImGui docking branch with this default layout:

- **Viewport** (center, large) — the simulation, rendered to a `RenderTexture2D` and displayed via `ImGui.ImageRenderTextureFit(...)`.
- **Scene Hierarchy** (left, vertical) — tree view of all entities grouped by `Hierarchy` parent.
- **Inspector** (right, vertical) — properties of selected entity.
- **Toolbar** (top, horizontal strip) — play/pause/step, time-scale slider, integrator dropdown, save/load buttons.
- **Asset Palette** (bottom, dockable) — body templates to drag into the viewport.
- **Diagnostics** (floating, toggleable) — ImPlot.NET graphs.
- **Console / Log** (floating, toggleable) — scrolling Serilog output piped through a custom sink, FPS, ms/frame, entity count.

5.4.3 Interactions

Input	Effect
Left-click body	Select it (sets <code>SelectionState</code> , fires <code>BodySelectedEvent</code>)
Left-click empty space	Deselect
Left-drag body	Move it (issues <code>MoveEntityCommand</code>)
Left-drag empty space	Box-select multiple bodies
Right-drag	Pan camera
Scroll wheel	Zoom (under cursor)
Drag from palette	Spawn body at drop position (issues <code>CreateEntityCommand</code>)
Right-click	Context menu (Create here, Paste, Focus camera, ...)
F	Focus camera on selected
Delete	Delete selected
Ctrl+D	Duplicate selected
Ctrl+Z / Ctrl+Y	Undo / redo
Ctrl+S	Save
Space	Pause/resume

5.4.4 Undo/Redo (Critical)

Implement the **command pattern** from day one. Every mutation goes through a `Command` :

```
public interface ICommand
{
    void Execute(Scene scene);
    void Undo(Scene scene);
    string Description { get; }

    // Returns true if this command merged into `last` (e.g. consecutive drags).
    bool TryMergeInto(ICommand last) => false;
}

public class CommandStack
{
    private readonly Deque<ICommand> _undo = new();
    private readonly Deque<ICommand> _redo = new();
    private const int MaxUndo = 200;

    public void Execute(Scene s, ICommand cmd) { /* ... */ }
    public void ExecuteMerging(Scene s, ICommand cmd) { /* ... merge logic ... */ }
    public bool CanUndo => _undo.Count > 0;
    public bool CanRedo => _redo.Count > 0;
    public void Undo(Scene s) { /* ... */ }
    public void Redo(Scene s) { /* ... */ }
}
```

Concrete commands needed:

- `CreateEntityCommand`
- `DestroyEntityCommand`
- `ModifyComponentFieldCommand<TComponent, TField>` (generic; uses `Action<TComponent, TField>` setter delegates to avoid reflection)
- `ReparentCommand`
- `RenameCommand`
- `BulkDeleteCommand` (for multi-select)

C# pattern: generic field-modification commands compose well with delegates.
`ModifyComponentFieldCommand<Transform, Vector2D<double>>(entity, (ref t, v) => t.Position = v, oldValue,`

`newValue)` . This avoids per-field command classes.

5.4.5 Inspector

- One ImGui section per component type attached to the selected entity.
- Editable fields per component: `ImGui.DragFloat2` for vectors (converting to/from `Vector2D<double>` on the boundary), `ImGui.ColorEdit4` for `Renderable.Color` , scientific-notation input for masses (`ImGui.InputDouble` with format `"%.3e"`).
- Mutations create `ModifyComponentFieldCommand<>` rather than directly editing the world.
- “Add Component” dropdown for attaching new components.

5.4.6 Implementation Notes

- **Selection ray-cast:** project mouse position into world space, find the entity whose visible radius contains the click point. With many bodies, use the quadtree (built in month 6) for fast selection.
- **Drag threshold:** don’t issue a `MoveEntityCommand` for a click that travels < 3 pixels; treat as just a select.
- **Drag-merge for undo:** when the user drags a slider for 5 seconds at 60fps, that’s 300 mutation events. Merge consecutive mutations on the same `(entity, field)` within a 500ms window into one undo entry. Without this, one slider-drag = 300 Ctrl+Z presses to undo.
- **ImGui docking gotchas:** pin the ImGui.NET version. The docking branch occasionally breaks layouts on `imgui.ini` files between versions.
- **ref mutation through Arch queries:** use `world.TryGet(entity, out Transform t)` for read-then-write patterns, then `world.Set(entity, t)` . Or use `ref` queries for in-place mutation. Don’t mix the two.

5.4.7 Deliverable Demo

Build a 5-body scenario from scratch in 60 seconds without touching code. Save it. Close the app. Reopen, load. Move a planet. Hit Ctrl+Z. Hit Ctrl+Y. Delete a body, undo. Multi-select two bodies, delete both, undo restores both.

5.4.8 Report Content

- UI architecture chapter: panel system, docking, separation of view from model.
- Command pattern writeup: design, alternatives considered (memento pattern), why this scales better.
- Interaction design discussion: how the click/drag/select state machine works, edge cases handled.

5.5 Month 5: Particle System

5.5.1 Goal

A real particle system — pool-allocated, batch-rendered, capable of 100k+ particles at 60 fps. Multiple visual presets that look genuinely good. **No GC pressure in the hot loop.**

5.5.2 Architecture

`Particle` struct, kept tight (cache-friendly):

```
[StructLayout(LayoutKind.Sequential, Pack = 4)]
public struct Particle
{
    public Vector2 Position;           // System.Numerics float vector is fine here
    public Vector2 Velocity;
    public Color ColorStart, ColorEnd;
    public float SizeStart, SizeEnd;
    public float Age, Lifetime;
    public float Rotation, AngularVel;
    public ushort EmitterId;
    public ParticleFlags Flags;       // [Flags] enum: Alive, AffectedByGravity, Additive
}
```

- `ParticlePool` — fixed-size `Particle[]` (e.g., 100,000) allocated once at startup. Dense-packed: alive particles always occupy `[0, _aliveCount)` . `Kill(i)` swaps with `[_aliveCount-1]` .

- `ParticleEmitter` component (already in catalog).
- `ParticleSystem` — per frame:
 1. For each emitter, spawn N particles based on rate and elapsed time.
 2. Update all live particles: integrate position, age, kill expired.
 3. Apply force fields if `AffectedByGravity` (reuses gravity solver — modular!).
- `ParticleRenderer` — single draw call (or two — additive vs alpha) per frame, instanced via raylib's `rlgl` low-level API.

C# memory note: `Particle` is a `struct`, stored in a `Particle[]` array. That means 100,000 particles is one allocation, ~5.6MB on the managed heap, never GC'd during the loop. Compare to using a class: 100,000 heap objects, ~16 bytes each plus header overhead, GC pressure on every spawn-and-die. Structs in arrays are mandatory for this.

5.5.3 Emitter Parameters

- **Shape:** point, circle, ring, cone, line (for asteroid belts).
- **Rate:** continuous (particles/sec), burst (N at once), curve over time.
- **Initial conditions:** speed range, angle range, lifetime range.
- **Visual:** color gradient over life, size curve over life, blend mode (additive / alpha / multiply).
- **Behaviors:** drag, turbulence (Perlin), curl noise, attractor/repulsor.
- **Physics flags:** affected by gravity, exerts gravity (only for special cases — usually no, since particles are visual).

5.5.4 Built-in Effect Presets

Each is a saved emitter config in JSON:

Preset	Description
<code>star_corona</code>	Slow upward-radiating hot particles around stars; additive blending; subtle pulse
<code>accretion_disk</code>	Ring emitter around BH; particles orbit + spiral inward; color shifts red→white→vanish at horizon
<code>comet_tail</code>	Trails behind low-mass fast bodies; particles drift away from nearest star (solar wind direction)
<code>collision_explosion</code>	Burst on <code>BodyCollisionEvent</code> ; debris with mass; participates in gravity briefly
<code>starfield_dust</code>	Parallax background layers; static; tiny dim points
<code>nebula_cloud</code>	Large slow curl-noise; sparse; additive
<code>tidal_stream</code>	On Roche-limit crossing; stretched stream of fragments along orbit direction
<code>supernova</code>	Massive burst with shockwave (radial velocity falloff); single-use

5.5.5 Particle Editor UI

Dedicated panel where you can:

- Pick a preset to edit (or create new).
- Tweak all parameters live in the viewport.
- See a small preview emitter running in a sub-viewport.
- Save preset to library.
- Apply preset to selected entity (attaches `ParticleEmitter`).

5.5.6 Implementation Notes

- **Batched rendering:** do NOT call `Raylib.DrawCircle` per particle. Build one big vertex buffer via `rlgl` (raylib's low-level rendering API exposed in Raylib-cs as `Rlgl`) with positions + colors + sizes; one draw call. With 100k particles you absolutely need this.
- **No allocations in update:** do not use LINQ, do not allocate `Particle` instances on the heap, do not boxing. Iterate the array with a `for` loop and `ref` accessors: `ref Particle p = ref pool[i];`.
- **Curl noise:** for swirly nebula motion, sample 2D Perlin noise twice with offset, take perpendicular gradient — gives divergence-free flow that looks “fluid-like.”
- **Sorting for transparency:** additive blending is order-independent (great), so most particles don't need sorting. If you do alpha blending, sort back-to-front by camera depth — but in 2D this is trivial.
- **Lifetime jitter:** randomize lifetime per particle (`lifetime = base + Random.Shared.NextDouble() * (max-min)`) so they don't die in synchronized waves.
- `Random.Shared` : use `Random.Shared.NextDouble()` instead of `new Random()` . The latter allocates and seeds from time-of-day, causing identical particles in tight loops. `Random.Shared` is thread-safe and free since .NET 6.

5.5.7 Deliverable Demo

Load `tidal_disruption.json` : a planet approaches a black hole, crosses Roche limit, fragments into a stretched stream, fragments orbit and spiral in, accretion disk glows brighter as mass falls in. Frame counter shows 60 fps with ~50k particles active. GC counter (visible in the diagnostics panel via `GC.CollectionCount(0)`) doesn't increment during play.

5.5.8 Report Content

- Particle system chapter: pool design, batched rendering via `Rlgl` , the curl noise math, performance benchmarks.
- Why a custom system rather than a library? (control, integration with your gravity solver, learning, GC-aware design).
- Performance numbers: 10k / 50k / 100k particles → ms/frame breakdown (update vs. render).
- *C#-specific discussion*: GC awareness, struct-array layout, why class-based particles would fail at this scale.

5.6 Month 6: Barnes-Hut + Performance Scaling

5.6.1 Goal

Direct summation maxes out around 500-1000 bodies on a typical laptop. Replace it with an $O(N \log N)$ Barnes-Hut tree and parallelize the force computation with `Parallel.For` . Now you can run 5,000-10,000 bodies in real-time.

5.6.2 Quadtree

Quadtree node layout:

```
public struct QuadNode
{
    public Vector2D<double> Center;      // center of this region
    public double HalfSize;              // half-width
    public Vector2D<double> Com;         // center of mass of contained bodies
    public double TotalMass;
    public int Child0, Child1, Child2, Child3; // NW, NE, SW, SE indices; -1 if none
    public int BodyIndex;                // -1 if internal node
    public int BodyCount;
}
```

Use a flat `List<QuadNode>` (or `QuadNode[]` resized as needed) with indices, not class instances — better cache behavior, no GC pressure on rebuild, easier to clear and reuse (`_nodes.Clear()` keeps capacity).

Build phase: insert each body; subdivide leaves on collision. Compute COM for each internal node bottom-up after build.

5.6.3 Barnes-Hut Force Calculation

For each body, traverse from root:

```
function compute_force(body, node):
    if node is leaf:
        if node.body != body: accumulate direct force from node.body
    else:
        d = distance(body.pos, node.com)
        if (node.size / d) < theta:
            accumulate force from node.com with node.total_mass
        else:
            for each child: compute_force(body, child)
```

The θ parameter controls accuracy/speed tradeoff:

- $\theta = 0 \rightarrow$ full direct sum (matches $O(N^2)$ result exactly).
- $\theta = 0.5 \rightarrow$ standard accuracy, $\sim 10\times$ speedup at $N=1000$.
- $\theta = 1.0 \rightarrow$ faster, visible accuracy loss.
- $\theta > 1.0 \rightarrow$ unphysical “blob” approximation; only for stress-testing scaling.

Expose θ as a UI slider.

Verification test (do not skip): at $\theta=0$, Barnes-Hut must produce forces *identical* to direct sum (modulo floating-point ordering). Write this as a unit test and protect it religiously. If it ever breaks, your tree is buggy.

5.6.4 Multithreading

In C# this is much easier than the C++ equivalent — the TPL handles thread pool management:

```
Parallel.For(0, bodies.Length, i =>
{
    var force = _tree.ComputeForce(bodies[i].Position, theta, G, softening);
    _forces[i] = force * bodies[i].Mass; // separate output array; no contention
});

// Single-threaded write-back to the ECS
for (int i = 0; i < bodies.Length; ++i)
    bodies[i].ForceAccumulator += _forces[i];
```

- Force computation per body is independent given a read-only tree $\rightarrow$ embarrassingly parallel.
- Tree *construction* stays single-threaded (parallelizing it correctly is complex; not worth it at our N).
- Use `BenchmarkDotNet` to verify you’re getting the speedup you expect; measure on 1/2/4/8 cores.

C# false-sharing note: writing `_forces[i]` from different threads is safe if `i` values are partitioned (which `Parallel.For` does for you). It’s unsafe if multiple threads write to elements within the same cache line of the same array. `Parallel.For` with its default partitioner gives each thread a contiguous range, avoiding this.

5.6.5 Visualization

- Toggle to render the quadtree subdivision overlay (lines for each node boundary). Looks impressive *and* helps debug. Examiners love seeing the algorithm visualized.
- Color nodes by depth, or by “approximated vs. recursed” during the current frame’s traversal.

5.6.6 Performance Comparison

- Benchmark scenario: `galaxy_collision.json` with 10,000 bodies (two rotating disks).
- Diagnostics panel shows ms/frame for current solver.
- Toggle Direct vs. Barnes-Hut, observe crossover.
- Plot ms/frame vs. N for both, log-log axes — should clearly show $O(N^2)$ vs. $O(N \log N)$ curves. Generate the data with `BenchmarkDotNet` for rigor; matplotlib (Python) for the actual chart.

5.6.7 Implementation Notes

- **Quadtree bounds:** compute the bounding box of all bodies each frame; root node covers it. Don’t fix root size — it’ll fail when bodies fly far.
- **Empty-region waste:** in N -body systems with extreme mass concentration, many tree levels will be sparse. This is fine; Barnes-Hut handles it gracefully.

- **Numerical stability:** when computing force on a body from a node, add a small softening parameter ϵ to the denominator: $F = Gm_1m_2 / (r^2 + \epsilon^2)^{3/2}$. Without it, near-singular forces blow up time-stepping.
- **List vs. array:** prefer `QuadNode[]` over `List<QuadNode>` for hot iteration; use `CollectionsMarshal.AsSpan(list)` if you must use a List, to skip the bounds-check on each index.
- **Tree reuse:** if bodies haven't moved much, you can reuse last frame's tree with updated COMs. Skip in v1; add as optimization if needed.

5.6.8 Deliverable Demo

Galaxy collision: 10,000 bodies in two rotating disks slowly spiral together over millions of simulated years. Smooth 60 fps. Toggle the quadtree visualization on — see the tree adapting to mass distribution every frame. Toggle Direct mode — frame rate drops to single digits. Toggle back.

Run a `BenchmarkDotNet` report and include the table in the report.

5.6.9 Report Content

- Algorithms chapter: quadtree construction (with complexity analysis: $O(N \log N)$ build, $O(N \log N)$ queries), Barnes-Hut convergence properties, error vs. θ .
- Multithreading: `Parallel.For` partitioning model, Amdahl's-law analysis, measured speedup vs. number of cores.
- Empirical performance: measured ms/frame at $N = 100, 500, 1k, 5k, 10k$ for both solvers; crossover point. BenchmarkDotNet's auto-generated tables make this trivial to present.

5.7 Month 7: Special Objects & Advanced Physics

5.7.1 Goal

Black holes feel like black holes. Close encounters don't blow up. The physics is genuinely sophisticated.

5.7.2 Adaptive Timestep

When bodies are close or fast, fixed dt is too coarse. Two options:

- **Global adaptive dt (RKF45)** — single timestep, error-estimated. Compute step at dt and at dt/2, compare; if error too large, halve dt. Simpler. Slows whole sim during close encounters.
- **Hierarchical timestep** — bodies in close encounters get sub-stepped while the rest of the sim runs at base dt. More complex, more impressive in the report.

Recommended: implement global adaptive (RKF45) first; mention hierarchical as future work.

5.7.3 Black Holes

- `BlackHole` component with Schwarzschild radius $r_s = 2GM/c^2$ (use an artificial `c` so r_s is visible at simulation scales — typical choice: `c = 1e7` in SI, giving stellar-mass $r_s \sim 0.02$ AU, visible at solar-system zoom).
- Bodies entering the event horizon are removed; mass and momentum transferred to the black hole.
- **First-order post-Newtonian correction:** add it as a *separate* `IForceField` rather than modifying `GravityField`. This keeps the modules orthogonal and lets you toggle it on/off live.

```
F_PN = F_Newton * 3GM/(rc^2)           (the *correction* term, added to F_Newton)
```

This reproduces correct perihelion precession for orbits near a BH. Mercury precesses 43 arcseconds per century in real life from GR; you can demo equivalent behavior.

- Visual: black disk for event horizon, glowing photon sphere ring at $1.5 r_s$, accretion disk emitter attached.
- **Stretch:** screen-space gravitational lensing shader (deferred to month 9 or as month 7 stretch).

5.7.4 Tidal Disruption

When a low-mass body crosses the Roche limit of a much heavier object:

- Roche limit: $d_{\text{Roche}} \approx 2.44 R \cdot (\rho_M / \rho_m)^{1/3}$ where R is the primary's radius and ρ are densities.
- On crossing: replace the body with $N=10-20$ fragments distributed along the orbit, conserving total mass and

momentum.

- Spawn `tidal_stream` particle effect.
- Fire `RocheLimitCrossedEvent` → audio cue, UI log entry.

5.7.5 Collision Modes

Per-scenario setting:

- **Elastic** — bounce with restitution coefficient (0-1).
- **Inelastic merger** — combine into one body, conserving mass and momentum, KE released as heat (visual flash).
- **Fragmentation** — high-energy collisions break both into N pieces.

Detected via simple distance check: if `|x_a - x_b| < r_a + r_b`, it's a collision. With many bodies, accelerate via the quadtree.

5.7.6 Test Particles

- Flag on `RigidBody`: `IsTestParticle = true`.
- Test particles feel gravity from massive bodies but don't exert it.
- Lets you simulate asteroid belts of 50k particles essentially free (Barnes-Hut handles their massless contribution trivially — exclude them from the tree, just query forces from it).
- Useful for ring system visualization, comet streams, debris fields.

5.7.7 Visualization Overlays

- Roche limit ring around heavy bodies (toggleable).
- Hill sphere boundary.
- Sphere of influence transitions (color the bodies by which primary dominates them).

5.7.8 Implementation Notes

- **Softening**: now critical for numerical stability with adaptive dt. $\epsilon \sim$ smaller of body radii.
- **Don't merge into BHs blindly**: bodies *crossing* the event horizon are absorbed; bodies *passing through* without crossing aren't (relevant for bodies on parabolic/hyperbolic orbits).
- **Conservation under merger**: `m_new = m_a + m_b`; `v_new = (m_a v_a + m_b v_b) / m_new`; `x_new = (m_a x_a + m_b x_b) / m_new`.
- **RKF45 implementation**: state has 4 doubles per body (x, y, vx, vy). For 1000 bodies that's 32KB — stays in L1/L2 cache. Use a `Span<double>` over a preallocated buffer; no `new double[]` per step.

5.7.9 Deliverable Demo

The viva centerpiece: stellar-mass black hole consumes a passing planet via tidal disruption. Planet stretches into a spaghettified stream, fragments swirl into the accretion disk, brightness flares as mass falls in. Pause, scrub the timeline back, replay. Switch off post-Newtonian correction → orbit visibly fails to precess. Switch on → precession matches theory.

5.7.10 Report Content

- Advanced physics chapter — adaptive timestep math, post-Newtonian correction derivation, Roche limit derivation, collision models compared.
- Honest scoping: what is and isn't simulated. Emphasize "first-order PN correction" — don't claim "general relativity."

5.8 Month 8: Tools, Polish, Scenarios, Export

5.8.1 Goal

It feels like a real product. A user who's never seen the code can build a stable solar system in 5 minutes. Tutorials, presets, video export.

5.8.2 Orbit Designer

The single most-impactful UX feature. When spawning a body:

1. User selects “Place in orbit around...” mode.
2. Clicks a parent body.
3. Drags out a radius from the parent.
4. Tool computes velocity vector for circular orbit: $v = \sqrt{GM/r}$ tangent to radius.
5. Optional eccentricity slider (0 = circular, $\rightarrow 1$ = elongated ellipse).
6. Click to commit; body is placed with computed velocity.

This single feature transforms usability from “frustrating” to “delightful.”

5.8.3 Predicted Orbit Overlay

For selected body, integrate forward in time without committing — render the path as a dashed line. Recompute every N frames (or on parameter change). Helps users understand what their setup will do before pressing play.

Implementation: clone the relevant subset of bodies into a “shadow” `Scene` (Arch supports world cloning via component copy), run M integrator steps, collect positions, render. ~200 lines of code; huge visual payoff.

5.8.4 Orbital Element Readout

For any selected body relative to chosen primary, display:

- Semi-major axis `a`
- Eccentricity `e`
- Period $T = 2\pi\sqrt{a^3/GM}$
- Periapsis $r_p = a(1-e)$ and apoapsis $r_a = a(1+e)$
- Specific orbital energy

Computed from current state vectors via standard orbital mechanics formulas (vis-viva, etc.). Update in real time as the body moves.

5.8.5 Lagrange Points

For any two-body subsystem, compute and display L1-L5:

- L4 and L5 are analytical (60° ahead/behind the smaller mass).
- L1, L2, L3 require numerical root-finding on a quintic. Use Newton-Raphson.
- Render as small x markers with labels.

Demo: Sun-Jupiter system $\rightarrow$ place a test particle at L4 $\rightarrow$ it stays put. Move it slightly $\rightarrow$ it oscillates around L4 (Trojan asteroid behavior).

5.8.6 Reference Frames

Camera modes:

- **World-fixed** (default).
- **Follow body** — camera tracks selected body.
- **Center-of-mass** of selection group.
- **Co-rotating frame** of two-body system — primary and secondary stay fixed; everything else moves around them. Lagrange points become visibly stationary. *Excellent* demo.

5.8.7 Time Controls

- Pause, single-step (advance one dt), full set of speed multipliers (0.1x, 0.5x, 1x, 10x, 100x, 1000x).
- **Reverse time** — only valid for symplectic integrators (Verlet, symplectic Euler). Disable for RK4 with a tooltip explaining why. *This is a great teaching feature.*

5.8.8 Export

- **Trajectory CSV** — per-body position and velocity over time, for external plotting in Python/matplotlib. Use `StreamWriter` with explicit flush per N rows; avoid memory ballooning.

- **Screenshot** — `Raylib.TakeScreenshot()` writes PNG.
- **Video export** — capture each frame to PNG sequence in a temp dir; ship a small script that calls `ffmpeg` to assemble. Or use `Process.Start("ffmpeg", "-y -framerate 60 ...")` from C# directly.
- **Scenario export** — already done via JSON save.

5.8.9 Scenario Library Expansion

Ship 12+ scenarios. Each has a “scenario card” with description, what to watch for, suggested time scale.

Scenario	Showcases
<code>solar_system.json</code>	Real planet data from JPL Horizons
<code>inner_planets.json</code>	Mercury precession with PN on/off
<code>trappist1.json</code>	Resonant chains
<code>binary_stars_with_planet.json</code>	Circumbinary orbits
<code>figure_eight.json</code>	Chenciner-Montgomery 3-body choreography
<code>pythagorean.json</code>	Classic chaotic 3-body
<code>lagrange_demo.json</code>	Sun-Jupiter with Trojans
<code>galaxy_collision.json</code>	10k-body Barnes-Hut showcase
<code>tidal_disruption.json</code>	Black hole eats planet
<code>kirkwood_gaps.json</code>	Asteroid belt resonance gaps
<code>kuiper_belt.json</code>	Test-particle ring around outer planet
<code>n_body_chaos.json</code>	50-body random scatter — pure chaos

5.8.10 Onboarding & Settings

- First-launch tutorial overlay: “Click here to pan. Scroll to zoom. Drag a body from the palette here. Press space to start the simulation.”
- Help menu with keyboard shortcut reference.
- “Tips” rotating in status bar.
- Settings panel: physics quality (timestep, integrator, softening ϵ), graphics (particle budget, trail length, blur strength, FPS cap), units (SI vs astronomical). Persist to `%APPDATA%/SolarSim/config.json` on Windows or `~/.config/SolarSim/config.json` on Linux — use `Environment.GetFolderPath(SpecialFolder.ApplicationData)`.

5.8.11 Deliverable Demo

Record a 2-minute showcase video, exported from the tool itself, that you’d be proud to put on YouTube. Cover: orbit designer, predicted overlay, scenario library, galaxy collision, tidal disruption, co-rotating frame, time-reverse.

5.8.12 Report Content

- Features chapter.
- Evaluation chapter — short user study with 3-5 classmates: give them the app cold, ask them to build a binary star system with two planets, time them, record what they struggle with.

5.9 Month 9: Stretch Features (Pick 2-3)

By this point you have a complete project. Use month 9 to add 2-3 stretch features that take the project from “good” to “memorable.”

5.9.1 Recommended Picks (by effort/impact ratio)

Feature	Effort	Impact	Recommendation
Lua scripting layer (NLua)	Medium	Very High	Strongly recommended — directly demonstrates the modularity claim
Black hole gravitational lensing shader	Medium	High	Visually striking, viva crowd-pleaser
Audio sonification	Low	Medium	Easy to add via raylib's audio; memorable
GPU compute force calculation (ILGPU)	High	High	Big speedup, but risky if you've never written compute kernels
Save state recording / replay	Medium	Medium	Different from time-reverse; powerful for analysis
Procedural galaxy generator	Low	Medium	Quick to implement, gives great demo scenarios
Native AOT publish	Low	Low	<code>dotnet publish -c Release -r linux-x64 --aot</code> for a single binary; impressive for “deployment” discussion in the report
Soft-body / spring networks	Medium	Medium	Niche extension
Magnetic fields / charged particles	Medium	Low	Cool but doesn't fit the “solar system” framing

5.9.2 Lua Scripting (Strong Recommendation)

Embed Lua via NLua. Expose:

- The Arch `World` (read-only access to bodies via wrapper).
- A force-field registration API: `register_force_field("my_force", function(entity_id, x, y, vx, vy, mass) return fx, fy end)`.
- A particle-emitter API.
- An event subscription API.

This single feature *proves* the modularity claim — you can load user-written `.lua` files at runtime and they slot into the simulation. Show the examiner a `.lua` file that adds a custom drag force; load it; watch it work. They will be impressed.

C# binding choice: NLua is the binding to use. It wraps KeraLua (a managed binding to native Lua), so it's fast. **MoonSharp** is an alternative pure-C# Lua interpreter (no native dependency) — easier to deploy but ~5-10× slower. Pick NLua unless deployment friction is a real problem; you can always switch later.

5.9.3 Lensing Shader

A screen-space displacement shader. For each pixel near a black hole:

- Compute impact parameter (perpendicular distance from BH-camera line).
- Compute deflection angle: $\theta \approx 4GM/(rc^2)$ (first-order GR).
- Sample background at displaced UV coordinates.

Raylib-cs exposes shader loading via `Raylib.LoadShader(vsPath, fsPath)`. Write the shader in GLSL ES 1.00 (`#version 100`) for portability. Approximate but very pretty.

5.9.4 Deliverable Demo

Whatever stretch features you picked, working and integrated.

5.9.5 Report Content

- Stretch features chapter.
 - Future work — list everything you didn't get to. Honesty here is *valued* in the report.
-

5.10 Month 10: Report, Polish, Defense Prep

5.10.1 Goal

Ship it.

5.10.2 Code

- Final bug-fixing pass.
- Run on a clean Windows VM and a clean Linux VM. Anything that requires “you have to install X first” needs to be fixed or documented.
- Run scenarios at length: leave the solar system simulation running for 8 hours; check no crashes, no NaNs in positions, no GC pauses > 5ms (use `dotnet-counters` to watch GC heap size).
- Code cleanup: `dotnet format` pass, remove dead code, finalize XML doc comments on public APIs.

5.10.3 Documentation

- **User manual** (PDF, illustrated).
- **Architecture document** (for examiners).
- **Code documentation** — XML doc comments on public APIs; generate HTML via DocFX.
- **README polish** — screenshots, GIF of the tool in action, build instructions verified on both platforms.

5.10.4 Report

You should have ~70% of the report drafted from monthly writing. Now:

- Integrate, edit for flow.
- Add diagrams (architecture, ECS data flow, Barnes-Hut tree, particle system pipeline).
- Performance benchmarks chapter with final numbers from BenchmarkDotNet runs.
- Evaluation chapter with user study results.
- Conclusion.
- Bibliography (you've been collecting references all year — now format them).

5.10.5 Defense Prep

Build a 10-minute demo script:

1. Open app, load `solar_system.json`. (30s)
2. Show orbit designer: spawn a planet around the Sun. (1m)
3. Switch to `galaxy_collision.json`. Show 10k bodies running smoothly. Toggle Barnes-Hut viz. (1.5m)
4. Switch to `tidal_disruption.json`. Run it. Pause at climax. (1.5m)
5. Show diagnostics panel. Switch integrator from Verlet to RK4. Show energy drift. (1.5m)
6. Show architecture diagram on slides. Reference modularity claim. (1m)
7. Show the Lua plugin demo (if implemented). (1m)
8. Q&A buffer. (2m)

Practice this five times. Time it. Have a backup pre-recorded video for every demo in case something breaks live.

5.10.6 Final Submission

- Source code (zipped or git URL).
- Built binaries for Windows + Linux (`dotnet publish -c Release -r win-x64 --self-contained` and `linux-x64`).
- Report PDF.
- Demo video (link or attached file).

- User manual.
- All scenario JSON files.

6. Risk Register

Risk	Likelihood	Impact	Mitigation
Floating-point precision at large scale	High	High	Doubles for sim via <code>Silk.NET.Maths.Vector2D<double></code> ; floating origin from month 1; never use <code>System.Numerics.Vector2</code> (float) for positions
GC pauses during particle-heavy scenarios	Medium	High	Struct-array pool from day one of month 5. Never allocate <code>Particle</code> instances. Monitor via <code>dotnet-counters</code>
Scope creep into 3D	Medium	Critical	Resist. 2D is plenty. Say no when tempted.
Particle system rendering bottleneck	High	High	Batched rendering via <code>R1g1</code> from day one of month 5. Never <code>DrawCircle</code> per particle
Barnes-Hut bugs (subtle and hard to spot)	High	High	Unit test against direct sum at small N; visualize tree; $\theta=0$ must match direct exactly
ImGui.NET docking branch flaky between versions	Medium	Medium	Pin a known-good NuGet version in <code>Directory.Packages.props</code> ; never auto-update mid-project
Save format breaks during iteration	Certain	Medium	Version field from day one; migration function per bump
Demo machine differs from dev machine	Certain	Medium	Test on clean Windows + clean Linux at month 10; or ship self-contained <code>dotnet publish</code> builds
Native dependency (raylib.dll / .so) fails to load	Medium	High	Raylib-cs ships natives in the NuGet for major platforms; test on minority targets (ARM Linux, macOS) if you care about them
Examiner asks “what’s novel?”	Certain	High	Have rehearsed answer (see §7)
Live demo breaks during viva	Medium	Critical	Pre-recorded backup video for every demo segment
You burn out around month 6	Medium	High	Time off scheduled in month 8.5 between core and stretch work
NLua/Lua integration eats month 9	Medium	Medium	Time-box. If not working in 5 days, drop it and pick something else from the list
CI breaks and you ignore it	High	Medium	Treat red CI as a P0 — fix same day always
.NET SDK auto-updates and breaks build	Medium	Medium	Pin <code>global.json</code> at repo root with <code>"sdk": { "version": "8.0.x", "rollForward": "latestPatch" }</code>

7. What Makes This Defendable as a Final-Year Project

You'll be asked, point-blank, "what's the contribution?" Have these answers rehearsed:

1. **Integration breadth.** Most open-source N-body simulators are either headless research code (REBOUND, NBODY6) or simple visualizers (web demos with hardcoded scenarios). Yours is a full interactive scene editor with scene hierarchy, undo/redo, orbit designer, predicted overlays, and 12 shipped scenarios.
2. **Architectural modularity (measurable).** How many lines change to swap the integrator? Three (one new file, one factory entry, one UI dropdown entry). To add a new force law? Five. To add a new particle behavior? One. You can quote actual diff sizes. *And* the modularity is enforced at the `.csproj` level: `SolarSim.Core` cannot reference Raylib-cs because the project file doesn't.
3. **Visual fidelity for a 2D simulator.** A custom particle system supporting 100k+ particles with curl-noise behaviors, accretion disks, tidal streams, comet tails — integrated with the same gravity solver — is not standard.
4. **Honest physics depth.** Four numerical integrators with empirical comparison; Barnes-Hut with measured speedup; first-order post-Newtonian correction; Roche-limit-driven tidal disruption. You went well past " $F=Gm_1m_2/r^2$ with Euler."
5. **GC-conscious C# design.** A particle system that allocates zero objects per frame at steady state. A quadtree that reuses its node buffer across frames. This is *not* typical C# code — and demonstrating you understood the GC well enough to bypass it earns marks in a CS curriculum.
6. **Plugin/extensibility layer (if Lua done).** Concrete demonstration that "modular" isn't a buzzword. User-written Lua files alter the simulation at runtime.

That's the thesis. Build toward it consciously every month and the project becomes hard to argue with.

8. Mapping to CS Curriculum Criteria

How the project content maps to typical final-year evaluation rubrics:

Curriculum Area	Where It Shows Up
Numerical methods	Integrator comparison, error analysis, stability, symplectic vs. non-symplectic, adaptive stepping (15-20 pages of report content)
Algorithms & data structures	Quadtree construction, Barnes-Hut traversal, complexity analysis with empirical verification, ring buffers for trails, struct-array pools for particles
Software engineering	Multi-project solution (Core / Render / UI separation enforced at the build level), test strategy (xUnit + FluentAssertions), CI, command pattern for undo/redo, central package management
Computer graphics	Camera math, world↔screen transforms, batched particle rendering via <code>R1gl1</code> , custom shaders (lensing, additive blending)
Concurrent / parallel programming	<code>Parallel.For</code> partitioning, parallel Barnes-Hut force calculation, Amdahl analysis, false-sharing considerations
Programming languages	C# language features used in anger: structs vs. classes, <code>ref</code> returns, <code>Span<T></code> , <code>[StructLayout]</code> , generic constraints, source generators (System.Text.Json), generic math types (Silk.NET.Maths)
Memory management	GC-conscious design: struct arrays for pools, <code>Random.Shared</code> , pre-allocated buffers, <code>ArrayPool<T></code> where applicable. Quantified by <code>dotnet-counters</code> measurements showing zero Gen-0 GCs during steady-state simulation
HCI	Designing the orbit-placement tool is a real interaction problem; reflective design discussion; user study
Domain knowledge (physics)	Kepler's laws, two-body analytical solution, conservation laws, post-Newtonian corrections, Roche limit, Schwarzschild radius

9. C#-Specific Notes for the Learner

Since you're using this project to *learn* C#, here are the concepts you'll naturally pick up by building this:

Month	C# concepts that come up naturally
1	<code>.csproj</code> and the SDK-style project format; <code>Directory.Build.props</code> / <code>Directory.Packages.props</code> ; multi-project solutions; nullable reference types; <code>using</code> directives; namespaces; basic Serilog usage
2	<code>interface</code> + virtual dispatch costs; <code>struct</code> vs <code>class</code> decisions; <code>ref</code> parameters and <code>ref</code> returns; generic methods; <code>xUnit</code> + <code>[Fact]</code> / <code>[Theory]</code>
3	<code>System.Text.Json</code> with source generators (<code>[JsonSerializable]</code>); <code>JsonConverter<T></code> for custom types; LINQ basics (use sparingly); <code>Path.Combine</code> and cross- platform paths
4	Generics and <code>where T : struct</code> constraints; delegate types; <code>Action<T></code> / <code>Func<T></code> / <code>EventHandler</code> ; ImGui.NET's interop quirks (passing <code>ref</code> for native arrays)
5	<code>[StructLayout]</code> and explicit memory layouts; <code>Span<T></code> / <code>ReadOnlySpan<T></code> ; <code>ref struct</code> ; iterating struct arrays without boxing; <code>[Flags]</code> enums
6	<code>Parallel.For</code> and <code>Task.Run</code> ; thread safety basics; <code>BenchmarkDotNet</code> attributes and benchmarking discipline; <code>Stopwatch.GetTimestamp()</code> for nanosecond timing
7	Numerical algorithm implementation in idiomatic C#; arithmetic precision concerns; <code>MathF</code> vs <code>Math</code>
8	<code>Process.Start</code> for spawning ffmpeg; <code>Environment.SpecialFolder</code> for cross-platform paths; settings persistence
9	P/Invoke (via NLua's wrapped natives, you'll see it but not write it); native interop concepts; shader programming
10	<code>dotnet publish</code> and runtime identifiers (RIDs); Native AOT considerations; XML doc comments; DocFX

If you finish this project, you'll have substantial C# experience across language features, framework idioms, deployment, and the .NET tooling ecosystem. The project naturally surfaces these concepts in context — far more durable than working through them via tutorials.

10. Final Note

The trap in projects like this is making them *look* bigger than they are. Don't claim "real black hole simulation" when you mean "point mass with an event horizon visual." Don't claim "general relativity" when you mean "first-order post-Newtonian correction." Examiners can smell overclaiming from a mile away.

Conversely, *do* sell the hard parts honestly:

"I implemented four numerical integrators and analyzed their energy conservation properties on chaotic three-body systems, finding that velocity Verlet outperforms RK4 over long timescales despite being lower order, due to its symplectic structure preserving phase-space volume. I implemented this in C# with care for the garbage collector — the particle subsystem allocates zero managed objects during steady-state simulation, sustaining 60 FPS with 100,000 active particles."

That sentence alone tells the examiner you understood something real, and that you adapted your design to the language. Build a project's worth of those sentences, and you have a defense.

Good luck. Build it well.